Member-only story

# How I Built a Local File Organizer MCP Server That Gives Claude Code Superpowers Over Your Filesystem


Shweta Lodha Follow 15 min read · Apr 22, 2026

      

*Turn your AI assistant into a real filesystem agent — one that can scan, categorize, organize, and undo changes to your local files. If you prefer a video over an article, then YouTube link is at the end of this article.*

There is a moment every developer has experienced: you open your Downloads folder, stare at 847 unnamed files with names like `Screenshot_2024-11-03_at_14.23.51.png`, `final_FINAL_v3_USE_THIS.docx`, and `untitled (1).zip`, and quietly close the window again.

You know what would help? Your AI assistant doing it for you.


But here is the problem. Claude, GPT, Gemini — they are all remarkably capable of reasoning about files. They just cannot *touch* them. Not your local files, anyway. They live in the cloud. Your Downloads folder does not.

That gap is exactly what the **Model Context Protocol (MCP)** was designed to close. And in this article, I am going to walk you through how I built a a quality *MCP* server — the **Local File Organizer** — that gives Claude Code six powerful tools to scan, categorize, move, and organize files on your local machine. With undo support. With dry-run mode. With a custom category system you can extend.

By the end, you will have a working *MCP* server you can register with *Claude Code* and immediately start using to tame that Downloads folder.

Let us get into it.

👉 **Free Download:** Your productivity guide

## What Even Is MCP?

Before we write a single line of code, let us make sure we
page about what *MCP* is and why it matters.

**Model Context Protocol** is an open standard that defines how *AI* models communicate with external tools and data sources. Think of it as the USB-C port for *AI* integrations.

> *Instead of every* AI *assistant inventing its own proprietary plugin format, MCP gives you one protocol that works across clients.*

The protocol works like this: your *MCP* server exposes a set of *tools* — each tool has a name, a description, and a typed input/output schema. The *AI* client (in our case, Claude Code) discovers those tools, understands what they do from the description, and calls them when appropriate. The server runs locally on your machine, which means it can do things the cloud model absolutely cannot: read files, move them around, write logs, access databases, call local APIs.

When Claude Code asks your *MCP* server to scan a directory, it sends a *JSON-RPC* message over stdio. Your server reads it, runs the *Python* code, and writes the result back over stdout. Claude then uses that result to decide what to do next.

That stdio transport is important and we will come back to it. It is also the reason one of our most important design constraints is: ***never write anything to stdout except MCP JSON-RPC messages.***

There are three things that make MCP genuinely powerful in practice:

1. **The AI discovers capabilities dynamically.** You do not hard-code “Claude can do X.” You tell *Claude* what tools exist and let it figure out when to use them.
2. **Tools are typed and described.** *Claude* reads the description of each tool and decides intelligently whether to call it. A good description is half the engineering.
3. **The server runs in your environment.** This is what makes local file access possible.

Now let us build something with it.

## What We Are Building

The **Local File Organizer MCP Server** exposes six tools to *Claude Code*:

scan_directory, categorize_files, organize_directory, move_file, undo_last_operation, get_categories

The server is built in Python using *FastMCP* , Pydantic v2 for typed I/O models, and a JSON-backed operation log that survives server restarts so that undo works even if *Claude Code* was closed between the organize and the undo.

Here is the project structure we are working toward:

```
LocalMCPServer/
├── server.py               # FastMCP app + tool registrations
├── __main__.py             # python -m local_file_organizer
├── config.py               # Category map, paths, load/save helpers
├── models.py               # Pydantic v2 I/O models
├── operation_log.py        # JSON-backed undo stack
├── tools/
│   ├── scan.py
│   ├── categorize.py
│   ├── organize.py
│   ├── move.py
│   ├── undo.py
│   └── categories.py
├── tests/
├── requirements.txt
├── pyproject.toml
└── .claude/
    └── settings.json       # MCP registration for Claude Code
```

Clean separation of concerns. The `server.py` handles MCP registration. The `tools/` package holds the actual business logic. Models live in one place. The config — category definitions and file paths — lives in its own module.

## Setting Up the Project

Start by creating a virtual environment and installing the dependencies:

```
python -m venv .venv
pip install -r requirements.txt
```

The `requirements.txt` lists two core dependencies:

```
mcp[cli]
pydantic>=2.0
```

`mcp[cli]` pulls in the *FastMCP* framework along with its *CLI* tooling. *Pydantic* v2 gives us the typed model layer. Everything else — `shutil`, `pathlib`, `uuid`, `json`, `logging` — is part of the *Python* standard library.

Once dependencies are installed, you can smoke-test the server:

```
python server.py
```

You should see a log line on stderr and then silence — the server is waiting for stdin input, which is exactly what *Claude Code* will provide. Press `Ctrl+C` to exit.

## The Models Layer: Getting Types Right First

One of the best decisions in this project was defining all the I/O types in `models.py` before writing any tool logic. Pydantic v2 gives us serialization, validation, and clear data contracts — and `.model_dump()` on every result means our tools return clean dicts that go straight back to Claude.

> If this resonated with you, I'd love to share Speakly with you — a product I built to convert your voice to

> text in an offline manner, no internet required. Give it a try https://shwetalodhajain.gumroad.com/l/speaklyv1!

Let us look at what we modeled.

## FileInfo — the atom of our system

```
class FileInfo(BaseModel):
    path: str
    name: str
    extension: str
    size_bytes: int
    size_human: str
    modified: str
    category: str = "Misc"
```

```
    @classmethod
    def from_path(cls, p: Path, category: str = "Misc") -> "FileInfo":
        stat = p.stat()
        size = stat.st_size
        return cls(
            path=str(p),
            name=p.name,
            extension=p.suffix.lower(),
            size_bytes=size,
            size_human=_human_size(size),
            modified=datetime.fromtimestamp(stat.st_mtime).isoformat(timespec="s
            category=category,
        )
```

The `from_path` classmethod is a factory that takes a `pathlib.Path`, calls `stat()` once, and builds the full model. We store both `size_bytes` (for sorting and filtering in code) and `size_human` (for Claude to read naturally). Lowercase extension ensures our category lookup is case-insensitive.

## Operation models — the backbone of undo

The undo system depends on two models:

```
class MoveRecord(BaseModel):
    """Single file move that can be reversed."""
    original_path: str
    new_path: str
    timestamp: str = Field(
        default_factory=lambda: datetime.now().isoformat(timespec="seconds")
    )
```

```
class OperationRecord(BaseModel):
    """A batch operation (organize or single move) stored for undo."""
    operation_id: str
    operation_type: str
    source_directory: str | None = None
    moves: list[MoveRecord]
    timestamp: str = Field(
        default_factory=lambda: datetime.now().isoformat(timespec="seconds")
    )
```

`MoveRecord` captures a single before/after path pair. `OperationRecord` wraps a list of those into a named, timestamped batch that can be pushed to and popped from the undo stack. The UUID-based `operation_id` makes each operation uniquely addressable.

## The Config Layer: Category Maps and File Paths

`config.py` does two things: it defines the default category-to-extension mapping, and it handles loading and saving custom category overrides.

```
DEFAULT_CATEGORIES: dict[str, list[str]] = {
    "Images": [...],
    "Documents": [...],
    "Videos": [...],
    "Audio": [...],
    "Code": [...],
    "Archives": [...],
    "Data": [...],
    "Misc": [],
}
```

The `Misc` category is intentionally left empty. It is the catch-all — any file whose extension is not recognized falls into Misc automatically. You can extend any category at runtime through the `get_categories` tool, and those extensions are persisted to `custom_categories.json`.

## The Operation Log: Undo That Survives Restarts

This is one of the most important architectural pieces in the project, and it is easy to underestimate why.

Here is the scenario: *Claude* organizes your Desktop at 9am. You close *Claude Code*. You come back at 2pm and realize you did not want that. You type `undo`. The server has been restarted in the meantime.

Without a persistent log, that undo is gone. With our JSON-backed stack, it is not.

```
_stack: list[OperationRecord] = []
_loaded: bool = False
```

The lazy-loading pattern via `_ensure_loaded()` means the log is only read from disk once per process lifetime — the first time any operation function is called. Every push and pop is immediately persisted via `_persist()`, which writes the entire stack back to disk as JSON.

You can call undo multiple times. Each call pops one record and reverses it. The stack grows forward in time and shrinks from the top — exactly what a proper undo stack should do.

## Building the Tools

Now for the interesting part. Each tool is a Python function in `tools/` and registered in `server.py` via a `@mcp.tool()` decorator. Let us walk through each one.

### Tool 1: scan_directory

This is the most fundamental tool. Before Claude can organize anything, it needs to see what is there.

```
def scan_directory(
    directory: str,
    recursive: bool = False,
    include_hidden: bool = False,
) -> ScanResult:
```

```
    base = Path(directory).expanduser().resolve()
    if not base.exists():
        raise ValueError(f"Directory does not exist: {directory}")
    if not base.is_dir():
        raise ValueError(f"Path is not a directory: {directory}")
```

```
    pattern = "**/*" if recursive else "*"
    file_infos: list[FileInfo] = []
    total_bytes = 0
    for p in sorted(base.glob(pattern)):
        ...
    return ScanResult(
        directory=str(base),
        total_files=len(file_infos),
        total_size_human=_human_size(total_bytes),
        files=file_infos,
    )
```

A few design choices worth noting here:

- `expanduser().resolve()` handles `~` expansion and relative paths, returning a clean absolute path every time.
- `glob("**/*")` vs `glob("*")` is toggled by the `recursive` parameter. Non-recursive is the safe default.
- Hidden files (dot-files) are skipped by default. On Windows and Mac, many system files start with a dot and you almost never want to move them.
- Errors on individual files are caught and logged but do not abort the scan. A single locked file should not prevent Claude from seeing the rest of the directory.

The registration in `server.py` pairs this logic with a clear description Claude can reason about:

```
@mcp.tool(
    name="scan_directory",
    description=(
        "List all files in a directory with metadata: size, file type, "
        "last-modified date, and inferred category. "
        "Set recursive=true to include subdirectories."
    ),
)
def scan_directory(directory: str, recursive: bool = False, include_hidden: bool
    try:
        return _scan(directory, recursive=recursive, include_hidden=include_hidd
    except ValueError as exc:
        return {"error": str(exc)}
    except Exception as exc:
        logger.exception("scan_directory unexpected error")
        return {"error": f"Unexpected error: {exc}"}
```

Notice the wrapper pattern: the MCP tool function in `server.py` calls the business logic function imported from `tools/`, catches exceptions, and returns clean dicts. The server never crashes. Errors become `{"error": "..."}` responses that Claude reads and handles gracefully.

### Tool 2: categorize_files

`categorize_files` builds on `scan_directory`. It calls the scan, then groups the results into `CategoryGroup` objects:

```
def categorize_files(
    directory: str,
    recursive: bool = False,
    include_hidden: bool = False,
) -> CategorizeResult:
    scan = scan_directory(directory, recursive=recursive, include_hidden=include
```

The ordering trick here is subtle: we iterate through `cats` (the canonical category order) to build groups, which means the result always comes back in a consistent, human-readable order: Images, Documents, Videos, Audio, Code, Archives, Data, Misc. Any custom categories the user has added appear at the end.

This is the tool *Claude* uses when you say "show me what is in my Downloads folder, organized by type." The response comes back with per-category counts, total sizes, and the full file list — enough for *Claude* to give you a useful summary and propose a plan.

### Tool 3: organize_directory (with dry-run)

This is the star of the show and the tool that requires the most care.

The design principle here: **never move a single file without the user understanding what will happen first.** The `dry_run` parameter enforces this at the API level. The default is `dry_run=True`, and the tool description explicitly tells Claude to always call with dry run first.

```
def organize_directory(
    directory: str,
    dry_run: bool = True,
    recursive: bool = False,
    include_hidden: bool = False,
```

```
    skip_categories: list[str] | None = None,
) -> OrganizeResult:
    skip = set(skip_categories or [])
    base = Path(directory).expanduser().resolve()
```

The `_unique_dest` helper deserves a mention. If `Documents/report.pdf` already exists when we try to move another `report.pdf` into Documents, it returns `Documents/report_1.pdf`, then `report_2.pdf`, and so on. No file ever gets silently overwritten.

The `skip_categories` parameter lets you preserve certain file types in place. Want to organize your Desktop but leave your Code files where they are? Pass `skip_categories=["Code"]`.

## Tool 4: move_file

For when you know exactly where one file should go:

```
def move_file(source: str, destination: str) -> MoveFileResult:
    src = Path(source).expanduser().resolve()
    dst = Path(destination).expanduser().resolve()
```

Note that `move_file` uses `shutil.move` rather than `Path.rename`. This matters on Windows: `rename` fails when source and destination are on different drives. `shutil.move` handles cross-drive moves correctly by copying then deleting.

Every successful move is pushed to the operation log, which means you can call `undo_last_operation` to get the file back.

## Tool 5: undo_last_operation

```
def undo_last_operation() -> UndoResult:
    record = pop_last_operation()
    if record is None:
        return UndoResult(success=False, operation_id="none", operation_type="no
                          files_restored=0, message="No operations to undo.")
```

The moves are reversed in order — we process them from most recent to oldest. For an organize operation with 200 files, this means each file goes back to exactly where it came from.

The `_cleanup_empty_dirs` call at the end removes the category subfolders (Images/, Documents/, etc.) if they are now empty. Without this, you would be left with a skeleton of empty folders after every undo.

### Tool 6: get_categories

The extensibility hook:

```
def get_categories(
    add_category: str | None = None,
    add_extensions: list[str] | None = None,
) -> CategoryMapResult:
    cats = load_categories()
```

You can ask *Claude* "add a category called Ebooks with `.epub` and `.mobi`" and this function normalizes the extensions (ensuring the leading dot), merges with existing custom categories, and persists to disk. Next time `organize_directory` runs, Ebooks is a first-class category.

## Putting It Together: The FastMCP Server

`server.py` is the entry point and the glue. It creates the *FastMCP* application, registers all six tools with their descriptions, and runs the server over stdio.

The `instructions` field is important. This is system-level guidance that *Claude* receives when it first connects to the server. Think of it as the README that *Claude* reads before it knows anything else about your tools. Telling *Claude* to always call `organize_directory` with `dry_run=true` first is not something you can enforce in code — but you can make it a strong instruction here.

Each tool registration wraps the business logic:

```
@mcp.tool(
    name="organize_directory",
    description=(
        "Auto-move files into categorised subfolders (Images/, Documents/, etc.)
        "ALWAYS call with dry_run=true first to preview the plan. "
        "Set dry_run=false to execute. "
        "Use skip_categories to leave certain types in place."
    ),
)
def organize_directory(
    directory: str,
```

```
    dry_run: bool = True,
    recursive: bool = False,
    include_hidden: bool = False,
    skip_categories: list[str] | None = None,
) -> dict:
    ...
```

The wrapping pattern is consistent across all tools: call the business logic, convert to dict, catch and return errors. No exception ever propagates to the *MCP* layer. This is critical — an unhandled exception in a tool call would crash the JSON-RPC exchange and potentially leave *Claude* in a confused state.

The server runs at the bottom:

```
if __name__ == "__main__":
    logger.info("Starting Local File Organizer MCP server (stdio)")
    mcp.run(transport="stdio")
```

`transport="stdio"` means the server reads requests from stdin and writes responses to stdout — the standard MCP local transport. This is why all logging uses `logging.StreamHandler(sys.stderr)`. If you accidentally `print()` anything to stdout, you corrupt the JSON-RPC stream and the server breaks silently.

## Registering with Claude Code

One of the nicest things about building a local *MCP* server for *Claude Code* is that registration is just a JSON file. Drop `.claude/settings.json` in your project directory:

```
{
  "mcpServers": {
    "local-file-organizer": {
      "command": "python",
      "args": ["server.py"],
      "cwd": "..\\LocalMCPServer"
    }
  }
}
```

Open *Claude Code* in that directory and the tools are automatically available. To verify: type `/mcp` in *Claude Code*. You should see `local-file-organizer` listed with all six tools.

That is the full integration.

No API keys.

No cloud accounts.

No webhooks.

Your tools, your filesystem, your AI.

## Seeing It in Action

Here are a few natural language prompts that demonstrate what the server enables:

**Scan a directory:**

> "Scan C:/Users/me/Downloads"

*Claude* calls `scan_directory`, gets back a list of files with metadata, and

summarizes: "You have 312 files totaling 4.3 GB. The largest is `video_export.mp4` at 1.8 GB, modified last week."

**Get a breakdown:**

> "Categorize all files in C:/Users/me/Downloads"

*Claude* calls `categorize_files`, gets back grouped results, and tells you: "Images: 87 files (234 MB). Documents: 43 files (89 MB). Archives: 12 files (1.2 GB). Code: 6 files (1.4 MB). Misc: 164 files (2.7 GB)."

**Preview the plan:**

> "Show me what organize_directory would do to C:/Users/me/Downloads (dry run)"

*Claude* calls `organize_directory` with `dry_run=True`, gets back the full list of planned moves, and presents them: "I would move 289 files. Here is the plan: 87 images into Images/, 43 documents into Documents/, ..."

**Execute:**

> "Now run it for real"

*Claude* calls `organize_directory` with `dry_run=False`. Files move. The operation is logged. *Claude* confirms: "Done. Moved 289 files. Operation ID: `a3f9b7c2-...` — you can undo this if needed."

**Undo:**

> "Actually undo that"

*Claude* calls `undo_last_operation`. All 289 files return to their original locations. Empty category folders are cleaned up. Claude confirms: "Restored 289 files."

**Add a custom category:**

> "Add a category called Ebooks with .epub and .mobi"

*Claude* calls `get_categories` with `add_category="Ebooks"` and `add_extensions=[".epub", ".mobi"]`. Those extensions are persisted to

`custom_categories.json`. Next time you organize, `.epub` and `.mobi` files go into an `Ebooks/` folder.

## Key Design Decisions Worth Remembering

Building this server surfaced a handful of decisions that are not obvious until you run into the problems they solve.

**Stderr-only logging.** This cannot be overstated. MCP over stdio uses stdout as the protocol channel. A single stray `print()` — or a library that logs to stdout — corrupts the JSON-RPC framing and the connection breaks with no clear error. Route every log line to stderr.

**Tools never raise exceptions.** The MCP layer expects either a valid result or a result with an `{"error": "..."}` field. An uncaught exception in a tool function propagates up in ways that are hard to debug. Catch everything at the server layer.

**Dry-run as a default, not an afterthought.** Making `dry_run=True` the default parameter value means Claude has to explicitly opt into executing operations. This shifts the default behavior toward safe previewing. Even if Claude forgets the instruction in the server description, the default protects you.

**The operation log must be persistent.** An in-memory undo stack is useless if the server can be restarted between operations. JSON persistence is simple, human-readable, and survives crashes. The lazy-load pattern means you only pay the disk read once per process.

**`shutil.move` over `Path.rename`.** On Windows, `rename` raises `OSError` when source and destination are on different drives (e.g., moving from `C:\Downloads` to `D:\Organized`). `shutil.move` handles this by doing a copy-then-delete when rename fails.

**Sorted glob output.** Calling `sorted(base.glob(pattern))` means the scan result is always in a deterministic order. This matters for tests and for making Claude's summaries consistent.

### Testing the Server

A production-quality server deserves tests. The `tests/` directory covers:

- `test_scan.py` — scan a temp directory with known files, assert correct metadata
- `test_categorize.py` — create files with specific extensions, assert correct groupings
- `test_organize.py` — dry run returns correct planned moves; live run actually moves files
- `test_undo.py` — organize then undo, assert files back in original locations
- `test_categories.py` — add custom category, verify it persists and loads

Run the full suite with:

```
pytest tests/ -v
```

All tests use `tmp_path` fixtures from pytest so they create and destroy temporary directories — no risk of accidentally touching real files during testing.

## What You Can Build on Top of This

The six tools we built are a foundation. Here are natural extensions:

**Duplicate file detection.** Add a `find_duplicates` tool that hashes files and reports groups with identical content. Perfect companion to organize — deduplicate before sorting.

**Watch mode.** A `watch_directory` tool that uses `watchdog` to monitor a directory and automatically categorize new files as they arrive. Combine with `organize_directory` for a live-sorting inbox.

**Smart naming.** A `rename_files` tool that takes a naming pattern and renames all files in a category. Think "rename all Images in my Desktop by date taken."

**Archive old files.** A `archive_old_files` tool that moves files not accessed in N days to an archive folder. Keep your active workspace clean automatically.

**Cloud sync awareness.** Pass known cloud sync directories (OneDrive, Dropbox) as skip targets so the organizer does not touch synced folders.

Each of these is one more `@mcp.tool()` registration and a function in `tools/`. The framework handles everything else.

## Wrapping Up

We built a local *MCP* server from scratch that gives *Claude Code* real, reversible control over your filesystem. Six tools. A persistent undo stack. Dry-run mode by default. A custom category system. And the whole thing runs over stdio with no cloud dependency.

The most important lesson from this build: **the *MCP* description is half the engineering.** *Claude* reads those tool descriptions and decides when and how to use them. A good description shapes *Claude's* behavior without any code. Write them carefully. Test what Claude actually does when it reads them.

The second lesson: **design for reversibility first.** Every destructive operation in this server — every move, every organize — is logged and can be undone. Before you write a tool that modifies state, ask yourself: what happens when

the user regrets this? Build the undo path before you build the happy path.

That messy Downloads folder does not stand a chance.

## Watch the Full Video Tutorial

If you want to see everything in action — the architecture, the folder structure, the workflow, the real example — I've created a full video that walks through the entire journey.

It's practical. It's visual. And it will help you build your first MCP Server from scratch.

👉 **Watch the full video here**

## Before you leave! 📣

1. **Clap** my article few times👏
2. **Buy** my product Speakly — Offline AI Voice-to-Text for Windows
3. **Follow** me on Medium and subscribe to get my latest article for Free🫶
4. **Subscribe** to the **YouTube channel**

AI | Artificial Intelligence | Python | Claude | LLM

**Written by Shweta Lodha**

1.94K followers · 346 following

I'm 8xC# Corner MVP, mentor, speaker, YouTuber & developer. YouTube: https://www.youtube.com/channel/UCQVOeT4i5nezvPG_xvIFzdQ Blog: http://www.shwetalodha.in

Follow